Câu 1 (6,5 điểm)

Giả sử bạn được giao thực hiện kiểm thử thành phần (component test) cho hàm được mô tả dưới đây. Hãy thiết kế và xây dựng số lượng test case thành phần (Unit Test case) tối thiểu cần thiết để đạt 100% statement coverage và 100% decision coverage. (Sử dụng mẫu của câu 2)

Một công ty tính tiền hoa hồng bán hàng cho nhân viên bằng hàm:

float fncSalesCommission(float rev, float ret, int nas)

trong đó rev là doanh thu, ret là giá trị hàng bị trả lại, nas là số trợ lý hỗ trợ bán hàng.

Hàm hoạt động như sau:

1. Đầu tiên, hàm kiểm tra dữ liệu đầu vào theo thứ tự: nếu rev < 0 thì trả về -1; nếu không, nếu ret < 0 thì trả về -1; nếu không, nếu nas < 0 thì trả về -1.

2. Nếu dữ liệu hợp lệ, hàm tính doanh thu tính hoa hồng: nr = rev – ret – 2,000,000 – nas × 1,000,000

3. Nếu nr ≤ 0 thì hàm trả về 0 (không có hoa hồng).

4. Ngược lại (nr > 0), tiền hoa hồng được tính theo các mức sau:

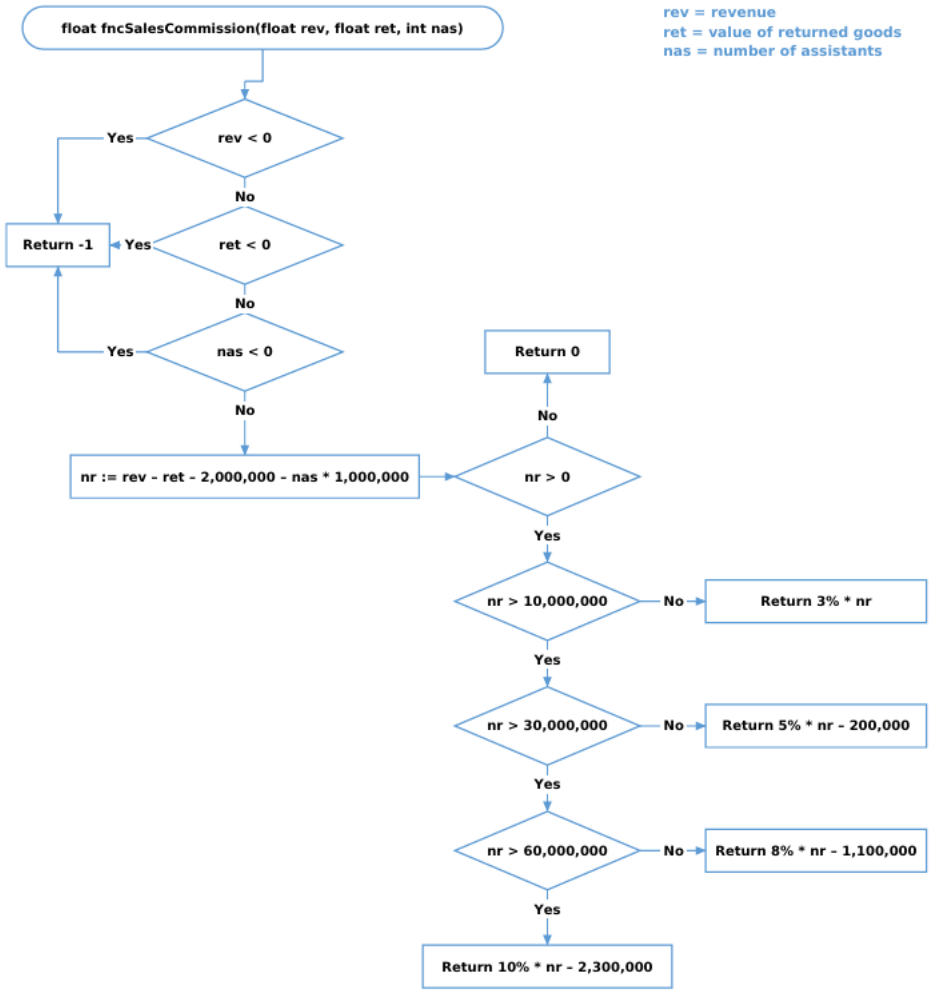
– Nếu nr ≤ 10,000,000: trả về 3% × nr

– Nếu 10,000,000 < nr ≤ 30,000,000: trả về 5% × nr – 200,000

– Nếu 30,000,000 < nr ≤ 60,000,000: trả về 8% × nr – 1,100,000

– Nếu nr > 60,000,000: trả về 10% × nr – 2,300,000

Lưu ý: các mức hoa hồng được kiểm tra lần lượt bằng các điều kiện nr > 10,000,000, nr > 30,000,000 và nr > 60,000,000



Câu 2 (3,5 điểm)

Giả sử bạn được giao thực hiện kiểm thử thành phần (component test) cho hàm Java dưới đây.

Hãy thiết kế và xây dựng số lượng test case thành phần (Unit Test case) tối thiểu cần thiết để đạt 100% statement coverage và 100% decision coverage.

Hàm calcShippingFee tính phí giao hàng, trong đó weight là khối lượng gói hàng (kg) và distance là quãng đường giao (km)

```java
public static double calcShippingFee(double weight, int distance) {
    if (weight <= 0) {
        return -1;
    }
    if (distance < 0) {
        return -1;
    }
    double fee = weight * 5000;
    if (distance > 50) {
        fee = fee + (distance - 50) * 1000;
    }
    if (weight > 20) {
        return fee * 0.9;
    }
    return fee;
}
```